

学号	1906410101	成绩	
----	------------	----	--



沈阳建筑大学

课程设计说明书

设计名称 综合课程设计 1

设计题目 叩丁狼限时抢购系统

设计时间 2022 年 5 月 30 日—6 月 17 日

学 院 信息与控制工程学院

专 业 计算机科学与技术

班 级 计算机 1901

姓 名 马明骏

指导教师 曹阳 王守金 韩子扬 郑新录 谢军

2022 年 6 月 16 日

综合课程设计 1 说明书

目 录

- 一、 题目简介
- 二、 功能说明
- 三、 程序框图
- 四、 程序清单
- 五、 运行结果
- 六、 设计体会
- 七、 参考文献
- 八、 教师评语

一、 题目简介

限时抢购是一个类似于淘宝双十一的在线商品抢购项目，项目的特点是并发量大，涉及大量的超卖,少卖等问题，也涉及到软件项目中缓存,分布式的定时任务等问题。其中的功能,涉及大量的高并发,高可用的设计思路 and 理念。

二、 功能说明

整体架构

微服务架构

本项目使用当下流行的微服务架构，通过 springcloud Alibaba 提供的生态组件来搭建项目。其中包括：

- 服务网关 spring cloud gateway
- 服务注册与发现中心兼配置中心 nacos
- 服务链路监控与熔断 sentinel
- Rpc 远程调用 Feign
- 分布式文件系统 zookeeper
- 微服务调度 elastic job

以及当下主流的关系数据库 MySQL 和缓存数据库 Redis。

服务网关

API 网关出现的原因是微服务架构的出现，不同的微服务一般会有不同的网络地址，而外部 客户端可能需要调用多个服务的接口才能完成一个业务需求，如果让客户端直接与各个微服务通信，会有以下的问题：

- 客户端会多次请求不同的微服务，增加了客户端的复杂性。
- 存在跨域请求，在一定场景下处理相对复杂。
- 认证复杂，每个服务都需要独立认证。
- 难以重构，随着项目的迭代，可能需要重新划分微服务。例如，可能将多个服务合并成一个或者将一个服务拆分成多个。如果客户端直接与微服务通信，那么重构将会很难实施。
- 某些微服务可能使用了防火墙 / 浏览器不友好的协议，直接访问会有一些困难。

以上这些问题可以借助 API 网关解决。API 网关是介于客户端和服务端之间的中间层，所有的外部请求都会先经过 API 网关这一层。也就是说，API 的实现方面更多的考虑业务逻辑，而安全、性能、监控可以交由 API 网关来做，这样既提高业务灵活性又不缺安全性。

网关中的核心概念如下：

- 路由：路由是网关最基础的部分，路由信息有一个 ID、一个目的 URL、一组断言和一组 Filter 组成。如果断言路由为真，则说明请求的 URL 和配置匹配
- 断言：Java8 中的断言函数。Spring Cloud Gateway 中的断言函数输入类型是 Spring5.0 框架中的 ServerWebExchange。Spring Cloud Gateway 中的断言函数允许开发者去定义匹配来自于 http request 中的任何信息，比如请求头和参数等。
- 过滤器：一个标准的 Spring webFilter。Spring cloud gateway 中的 filter 分为两种类型的 Filter，分别是 Gateway Filter 和 Global Filter。过滤器 Filter 将会对请求和响应进行修改处理。

客户端发送请求给网关，网关 HandlerMapping 判断是否请求满足某个路由，满足就发给网关的 WebHandler。这个 WebHandler 将请求交给一个过滤器链，请求到达目标服务之前，会执行所有过滤器的 pre 方法。请求到达目标服务处理之后再依次执行所有过滤器的 post 方法。

秒杀业务

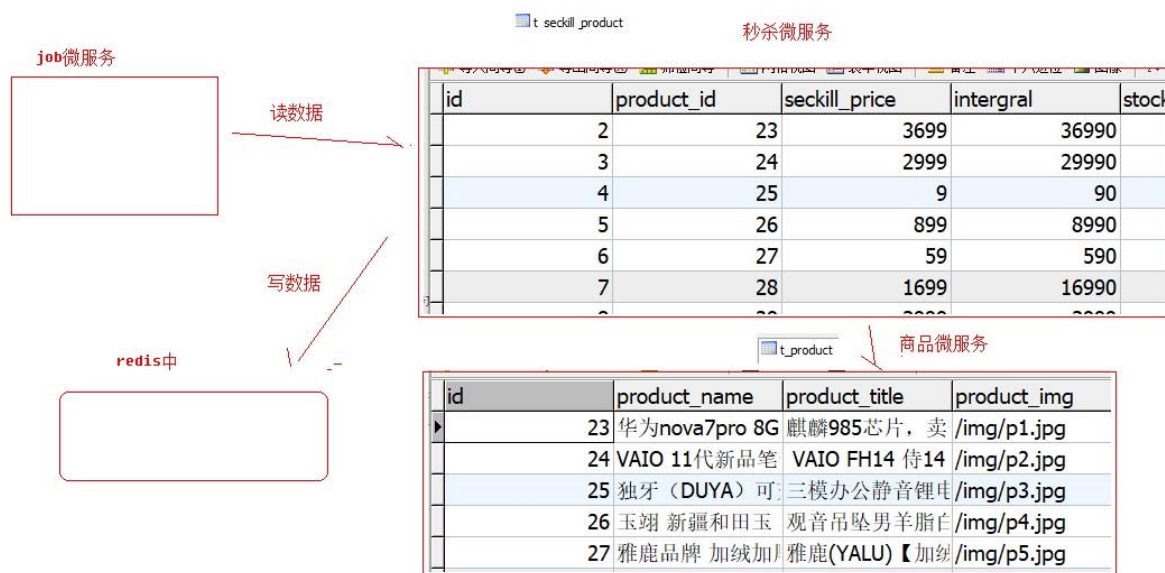
秒杀流程

用户点击秒杀按钮后，会发送一个 MQ 的消息，同时把自己的信息，存储到 WebSocket 长连接服务中，MQ 的客户端接收到消息后，执行表级操作，如果操作成功就发送延时消息，操作失败发送失败消息，无论成功还是失败，会发送结果消息给 WebSocket 服务，然后找到客户端的长连接对象，发结果发送给前端。

秒杀商品同步

因为秒杀开始后访问量会剧增，数据库无法承受过高的访问量需要提前将秒杀商品的信息保存到 redis 服务器中。这个需要定时任务来触发，比如每天 00:10 触发程序的执行，从数据库中把数据读取到 redis 中。进行今天的秒杀操作。

由于同样的服务会部署多个节点，多个节点的定时任务代码可能同时启动。将同样的事情做了多次，解决方案是使用分布式锁或者任务拆分并发执行在这里使用ElasticJob 作为



秒杀限流

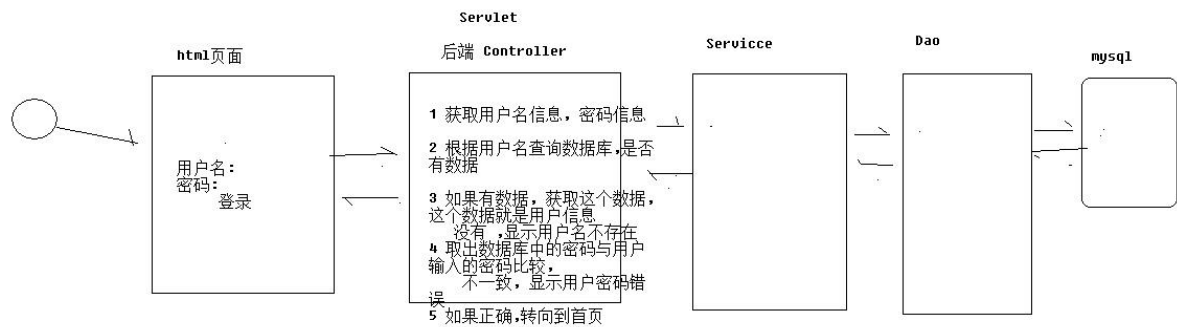
秒杀具有瞬间高并发的特点，针对这一特点，必须要做限流 + 异步 + 缓存（页面静态化） + 独立部署。

限流方式：

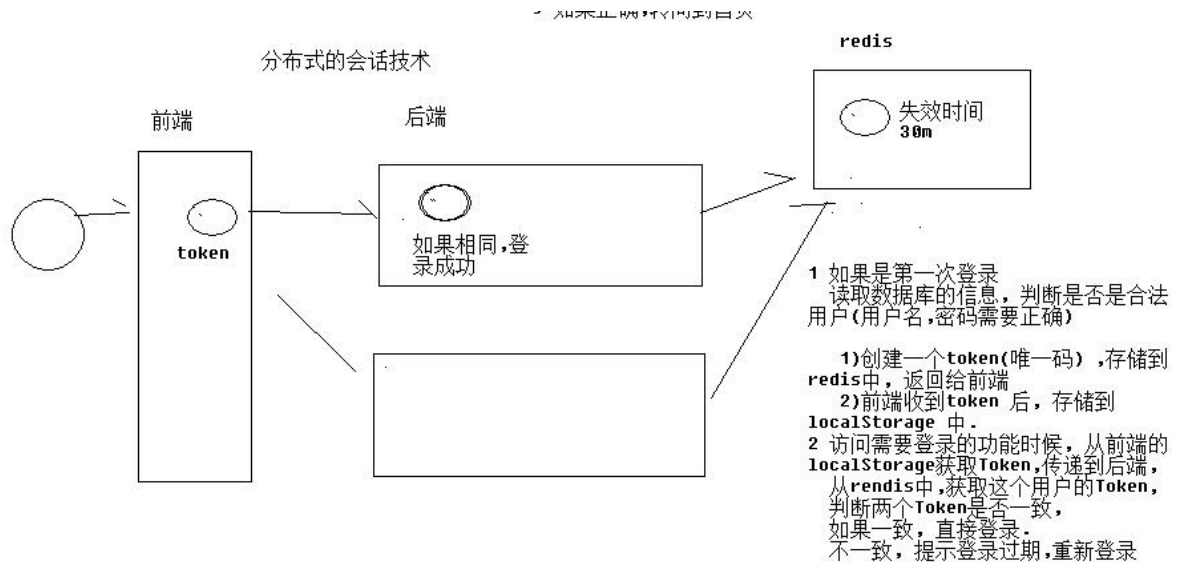
1. 前端限流，一些高并发的网站直接在前端页面开始限流，例如：小米的验证码设计
2. nginx 限流，直接负载部分请求到错误的静态页面：令牌算法 漏斗算法
3. 网关限流，限流的过滤器
4. 代码中使用分布式信号量
5. mq 限流（能者多劳：chanel.basicQos(1)），保证发挥所有服务器的性能。

登录业务

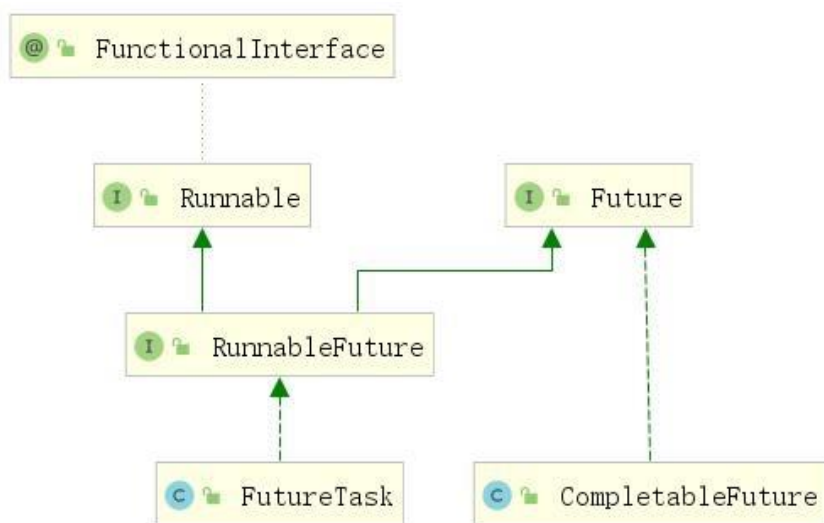
传统的登录方式



微服务登录方式



异步请求



业务场景：查询商品详情页的逻辑比较复杂，有些数据还需要远程调用，必然需要花费更多的时间。

假如商品详情页的每个查询，需要如下标注的时间才能完成 那么，用户需要 5.5s 后才能看到商品详情页的内容。很显然是不能接受的。 如果有多个线程同时完成这 6 步操作，也许只需要 1.5s 即可完成响应。

`Future` 是 Java 5 添加的类，用来描述一个异步计算的结果。你可以使用 `isDone` 方法检查计算是否完成，或者使用 `get` 阻塞住调用线程，直到计算完成返回结果，你也可以使用 `cancel` 方法停止任务的执行。

虽然 `Future` 以及相关使用方法提供了异步执行任务的能力，但是对于结果的获取却是很不方便，只能通过阻塞或者轮询的方式得到任务的结果。阻塞的方式显然和我们的异步编程的初衷相违背，轮询的方式又会耗费无谓的 CPU 资源，而且也不能及时地得到计算结果。

很多语言，比如 Node.js，采用回调的方式实现异步编程。Java 的一些框架，比如 Netty，自己扩展了 Java 的 `Future` 接口，提供了 `addListener` 等多个扩展方法；Google guava 也提供了通用的扩展 `Future`；Scala 也提供了简单易用且功能强大的 `Future/Promise` 异步编程模式。

在 Java 8 中，新增加了一个包含 50 个方法左右的类: `CompletableFuture`，提供了非常强大的 `Future` 的扩展功能，可以帮助我们简化异步编程的复杂性，提供了函数式编程的能力，可以通过回调的方式处理计算结果，并且提供了转换和组合 `CompletableFuture` 的方法。`CompletableFuture` 和 `FutureTask` 同属于 `Future` 接口的实现类，都可以获取线程的执行结果。

消息队列

大多应用中，可通过消息服务中间件来提升系统异步通信、扩展解耦能力
消息服务中两个重要概念：

消息代理（message broker）和目的地（destination）

当消息发送者发送消息以后，将由消息代理接管，消息代理保证消息传递到指定目的

3. 消息队列主要有两种形式的目的地

1. 队列（queue）：点对点消息通信（point-to-point）

2. 主题（topic）：发布（publish）/订阅（subscribe）消息通信

4. 点对点式：

- 消息发送者发送消息，消息代理将其放入一个队列中，消息接收者从队列中获取消息内容，消息读取后被移出队列

- 消息只有唯一的发送者和接受者，但并不是说只能有一个接收者

5. 发布订阅式：

- 发送者（发布者）发送消息到主题，多个接收者（订阅者）监听（订阅）这个主题，那么就会在消息到达时同时收到消息

6. JMS（Java Message Service）JAVA 消息服务：

- 基于 JVM 消息代理的规范。ActiveMQ、HornetMQ 是 JMS 实现

7. AMQP（Advanced Message Queuing Protocol）

- 高级消息队列协议，也是一个消息代理的规范，兼容 JMS

支付

1、公钥私钥 公钥和私钥是一个相对概念 它们的公私性是相对于生成者来说的。一对密钥生成后，保存在生成者手里的就是私钥，生成者发布出去大家用的就是公钥

2、加密和数字签名

加密是指：

我们使用一对公私钥中的一个密钥来对数据进行加密，而使用另一个密钥来进行解密的技术。

公钥和私钥都可以用来加密，也都可以用来解密。

但这个加解密必须是一对密钥之间的互相加解密，否则不能成功。

加密的目的是：

为了确保数据传输过程中的不可读性，就是不想让别人看到。

签名：

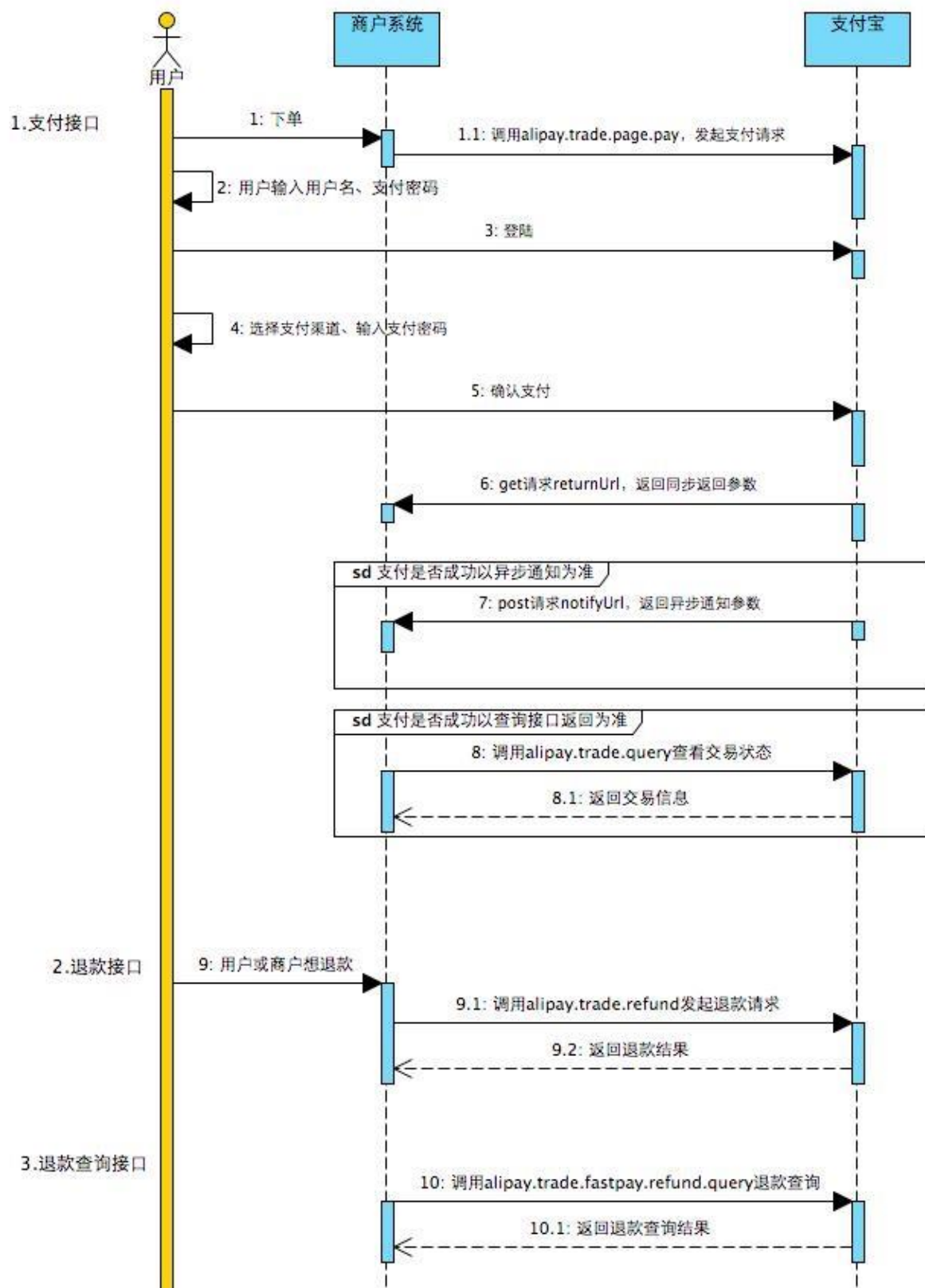
给我们将要发送的数据，做上一个唯一签名（类似于指纹）

用来互相验证接收方和发送方的身份；

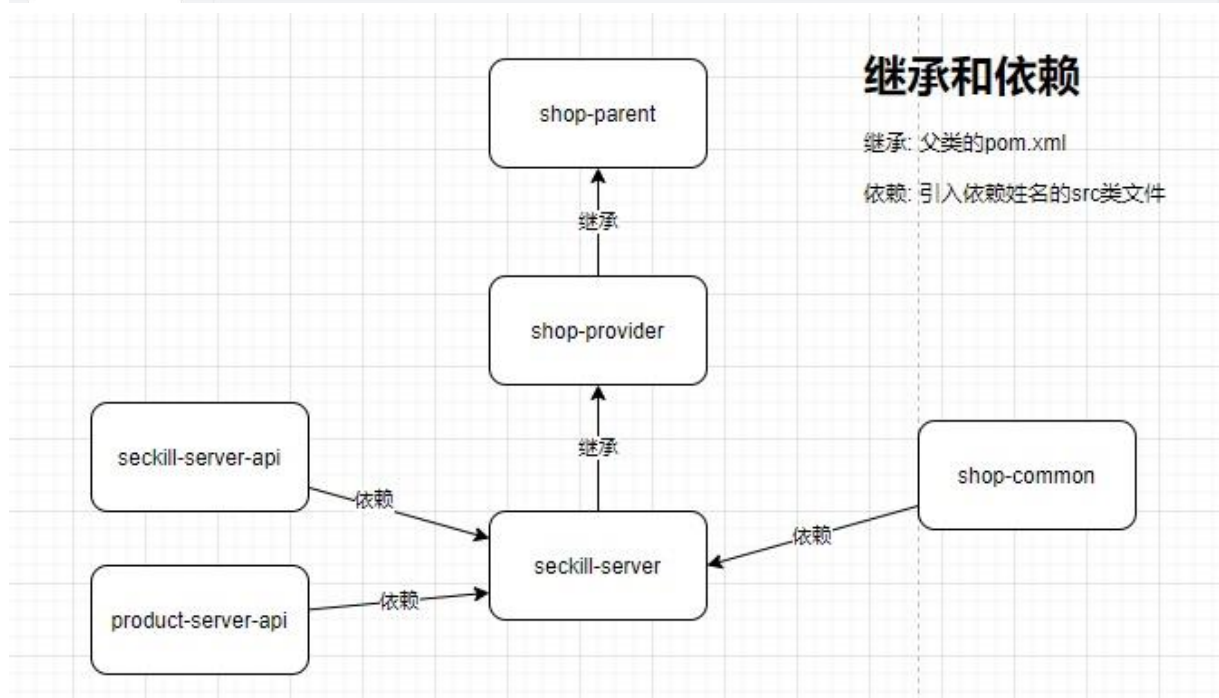
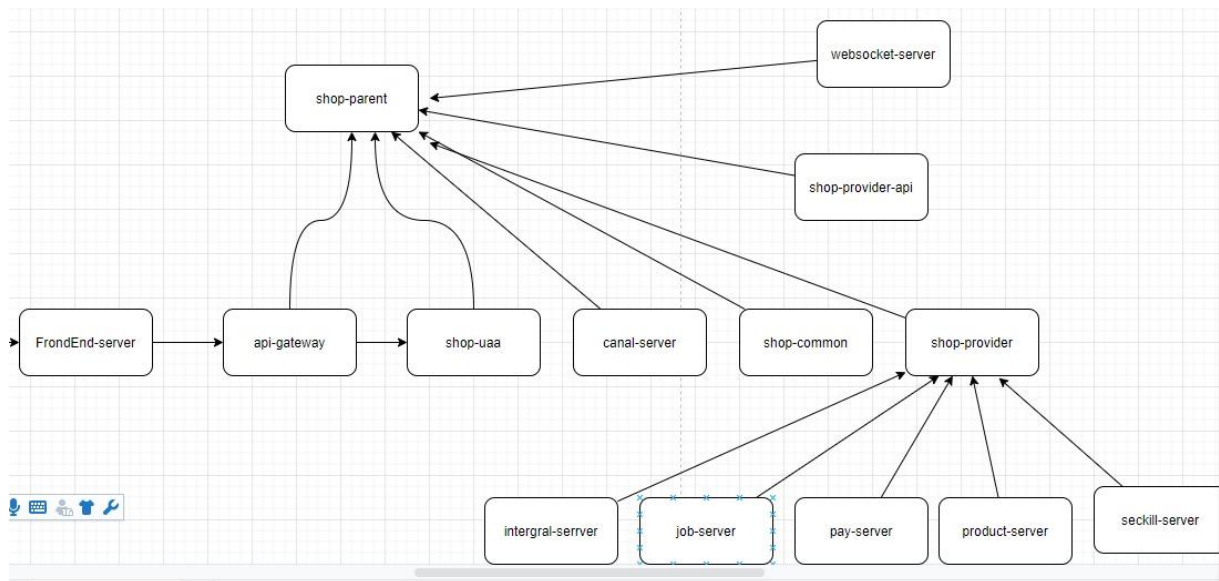
在验证身份的基础上再验证一下传递的数据是否被篡改过。因此使用数字签名可以用来达到数据的明文传输。

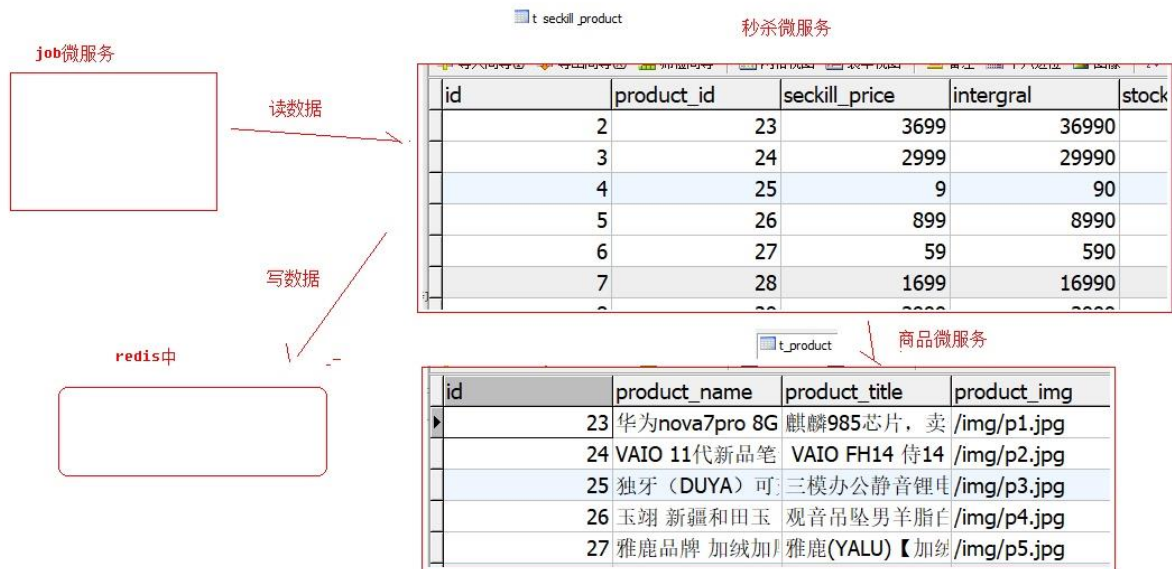
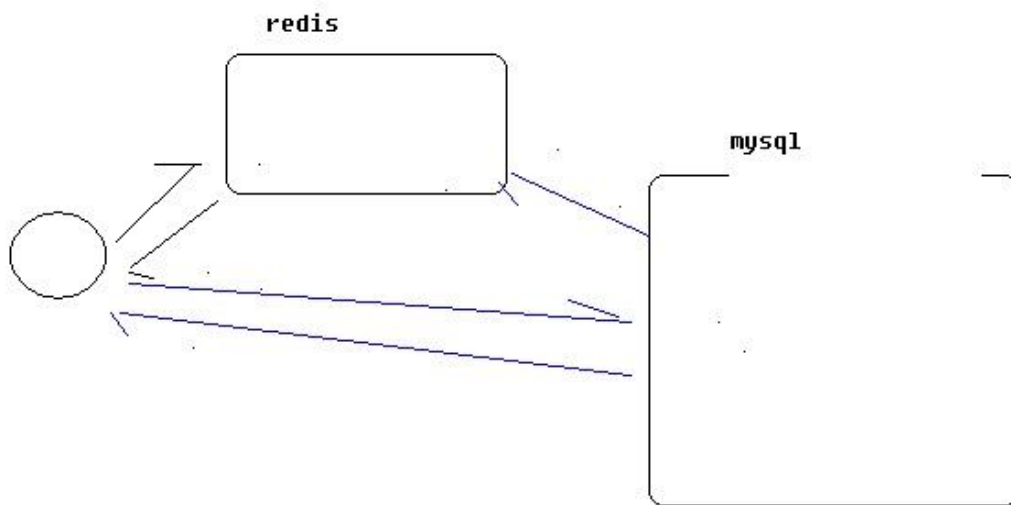
验签：

支付宝为了验证请求的数据是否商户本人发的，商户为了验证响应的数据是否支付宝发的。



三、 程序框图





四、 程序清单

▼	flash_sale	D:\OLD\建筑大学-2022实训\workspace\flash_sale	父工程
▶	frontend-server	前端项目	
▼	shop-parent	后端项目	
▶	api-gateway	网关	
▶	canal-client	同步框架	
▶	shop-common	通用功能项目	
▼	shop-provider	业务服务父工程	
▶	integral-server	积分服务	
▶	job-server	任务服务	
▶	pay-server	支付服务	
▶	product-server	产品服务	
▶	seckill-server	秒杀服务	
	m	pom.xml	1
▼	shop-provider-api	业务项目公共部分的父工程	
▶	integral-api	积分项目的共享部分	1
▶	pay-api	支付服务的共享	
▶	product-api	产品服务的共享	1
▶	seckill-api	秒杀服务的共享	
	m	pom.xml	1
▶	shop-uaa	用户服务	
▶	websocket-server	长链接服务	
▶	配置文件		
	m	pom.xml	
▶	图片		

秒杀服务

```
@RestController
@RequestMapping("/seckillProduct")
@Slf4j
public class SeckillProductController {
    @Autowired
    private ISeckillProductService seckillProductService;

    @RequestMapping("/selectSeckillProduct")
    public Result<List<SeckillProductVo>> selectSeckillProduct(){
        return Result.success(seckillProductService.selectSeckillProduct());
    }

    @RequestMapping("/queryByTime")
    public Result<List<SeckillProductVo>> queryByTime(@RequestParam Integer time){
        if(StringUtils.isEmpty(time)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(seckillProductService.queryByTime(time));
    }

    @RequestMapping("/find")
    public Result<SeckillProductVo> find(@RequestParam Integer time,@RequestParam Long seckillId){
        if(StringUtils.isEmpty(time) || StringUtils.isEmpty(seckillId)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(seckillProductService.find(time,seckillId));
    }
}
```

```
@FeignClient(name = "seckill-service",fallback = SeckillProductFeignFallback.class)
public interface SeckillProductFeignApi {
    cleye, 2022/6/13, 21:00 • 秒杀1

    @RequestMapping("/seckillProduct/selectSeckillProduct")
    Result<List<SeckillProductVo>> selectSeckillProduct();
}
```

```
public class SeckillCodeMsg extends CodeMsg {
    cleye, 2022/6/7, 14:42 • init
    private SeckillCodeMsg(Integer code, String msg) { super(code,msg); }

    public static final SeckillCodeMsg SECKILL_STOCK_OVER = new SeckillCodeMsg( code: 500201, msg: "您来晚了, 商品已经被抢购完毕.");
    public static final SeckillCodeMsg REPEAT_SECKILL = new SeckillCodeMsg( code: 500202, msg: "您已经抢购到商品了, 请不要重复抢购");
    public static final SeckillCodeMsg SECKILL_ERROR = new SeckillCodeMsg( code: 500203, msg: "秒杀失败");
    public static final SeckillCodeMsg CANCEL_ORDER_ERROR = new SeckillCodeMsg( code: 500204, msg: "超时取消失败");
    public static final SeckillCodeMsg PAY_SERVER_ERROR = new SeckillCodeMsg( code: 500205, msg: "支付服务繁忙, 稍后再试");
    public static final SeckillCodeMsg REFUND_ERROR = new SeckillCodeMsg( code: 500206, msg: "退款失败, 请联系管理员");
    public static final SeckillCodeMsg INTERGRAL_SERVER_ERROR = new SeckillCodeMsg( code: 500207, msg: "操作积分失败");
}
```

用户服务

```
@Component
@Setter@Getter
@RefreshScope
@Slf4j
public class UserCacheJob implements SimpleJob {    cleye, 2022/6/7, 14:42 * init
    @Value("${jobCron.userCache}")
    private String cron;
    @Autowired
    private StringRedisTemplate redisTemplate;
    @Override
    public void execute(ShardingContext shardingContext) { doWork(); }
    private void doWork() {
        Calendar calendar = Calendar.getInstance();
        calendar.add(Calendar.DATE, amount: -7);
        //获取7天前的日期
        Long max = calendar.getTime().getTime();
        String userZSetKey = JobRedisKey.USER_ZSET.getRealKey("");
        String userHashKey = JobRedisKey.USER_HASH.getRealKey("");
        Set<String> ids = redisTemplate.opsForZSet().rangeByScore(userZSetKey, v: 0, max);
        //删除7天前的用户缓存数据
        if(ids.size()>0){
            redisTemplate.opsForHash().delete(userHashKey,ids.toArray());
        }
        redisTemplate.opsForZSet().removeRangeByScore(userZSetKey, v: 0,calendar.getTime().getTime());
    }
}
```

```
@RestController
@RequestMapping("/token")
public class LoginController {
    @Autowired
    private IUserService userService;
    @PostMapping
    public Result<UserResponse> login(@RequestBody UserLogin userLogin, HttpServletRequest request) {
        /**
         * 获取用户IP,因为微服务的请求是网关转发过来的.
         * 所以request.getRemoteAddr()获取到的是网关的IP
         * 我们需要在网关中获取到真实IP,然后放入到请求头中.
         * 在微服务中通过获取请求头从而获取到真实的客户端IP
         */
        String ip = request.getHeader(CommonConstants.REAL_IP);
        String token = request.getHeader(CommonConstants.TOKEN_NAME);
        //进行登录,并将这个token返回给前台
        UserResponse userResponse = userService.login(userLogin.getPhone(),userLogin.getPassword(),ip,token);    cleye, 2022/6/7, 14:42 * init
        return Result.success(userResponse);
    }
}
```

```

spring:  cleye, 2022/6/7, 14:42 * init
  application:
    name: uaa-service
  cloud:
    nacos:
      config:
        server-addr: 202.182.125.38:8848 #nacos中心地址
        file-extension: yaml # 配置文件格式
        shared-configs:
          - data-id: redis-config-dev.yaml
          - data-id: rocketmq-config-dev.yaml
          - data-id: nacos-discovery-config-dev.yaml
      profiles:
        active: dev # 环境标识

```

```

public interface UserMapper {  cleye, 2022/6/7, 14:42 * init
    /**
     * 根据用户手机号码查询用户登录信息对象
     * @param phone
     * @return
     */
    UserLogin selectUserLoginByPhone(Long phone);

    /**
     * 根据用户手机号码查询用户的基础信息
     * @param phone
     * @return
     */
    UserInfo selectUserInfoByPhone(Long phone);

    /**
     * 插入登录日志
     * @param loginLog
     * @return
     */
    int insertLoginLong(LoginLog loginLog);
}

```

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="cn.wolfcode.mapper.UserMapper" >
  <insert id="insertLoginLong" useGeneratedKeys="true" keyProperty="id" >
    insert into t_login_log(phone,login_ip,login_time,state) values(#{phone},#{loginIp},#{loginTime},#{state})
  </insert>
  <select id="selectUserLoginByPhone" resultType="cn.wolfcode.domain.UserLogin" >
    select * from t_user_login where phone = #{phone}
  </select>
  <select id="selectUserInfoByPhone" resultType="cn.wolfcode.common.domain.UserInfo" >
    select * from t_user_info where phone = #{phone}
  </select>
</mapper>
cleye, 2022/6/7, 14:42 * init

```

商品服务

```
@RestController
@RequestMapping("/product")
@Slf4j
public class ProductController {    cleye, 2022/6/7, 14:42 • init
    @Autowired
    private IProductService productService;

    @RequestMapping("/findProductById")
    public Result<Product> findProductById(@RequestParam Long productId){
        if (StringUtils.isEmpty(productId)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(productService.findProductById(productId));
    }
}
```

```
@RestController
@RequestMapping("/product")
@Slf4j
public class ProductController {    cleye, 2022/6/7, 14:42 • init
    @Autowired
    private IProductService productService;

    @RequestMapping("/findProductById")
    public Result<Product> findProductById(@RequestParam Long productId){
        if (StringUtils.isEmpty(productId)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(productService.findProductById(productId));
    }
}
```

```
9 → public interface ProductMapper {    cleye, 2022/6/7, 14:42 • init
10  /**
11   * 根据用户传入的id集合查询商品对象信息
12   * @param ids
13   * @return
14   */
15 → List<Product> queryProductByIds(@Param("ids") List<Long> ids);
16
17 → Product queryProductById(Long productId);
18 }
```

```

<?xml version="1.0" encoding="UTF-8" ?> cleye, 2022/6/7, 14:42 • init
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="cn.wolfcode.mapper.ProductMapper" >
  <select id="queryProductByIds" resultType="cn.wolfcode.domain.Product" >
    select id,product_name,product_title,product_img,product_detail,product_price from t_product
    <where>
      id in
      <foreach item="item" collection="ids" separator="," open="(" close=")" >
        #{item}
      </foreach>
    </where>
  </select>
  <select id="queryProductById" resultType="cn.wolfcode.domain.Product">
    select id,product_name,product_title,product_img,product_detail,product_price
    from t_product where id=#{productId}
  </select>
</mapper>

```

```

public interface IProductService { cleye, 2022/6/7, 14:42 • init
    Product findProductById(Long productId);
}

```

```

@Service
public class ProductServiceImpl implements IProductService {
    @Autowired(required = false)
    private ProductMapper productMapper;

    @Override
    public Product findProductById(Long productId) { cleye, Yesterday • 秒杀2
        return productMapper.queryProductById(productId);
    }
}

```

网关服务

```
/**
 * 定义全局过滤器，功能如下：
 * 1.把客户端真实IP通过请求头的方式传递给微服务
 * 2.在请求头中添加FEIGN_REQUEST的请求头，值为0，标记请求不是Feign调用，而是客户端调用
 * 3.刷新Token的有效时间
 */
@Component
public class CommonFilter implements GlobalFilter {    cleye, 2022/6/7, 14:42 * init
    @Autowired
    private StringRedisTemplate redisTemplate;
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        /**
         * pre拦截逻辑
         * 在请求去到微服务之前，做了两个处理
         * 1.把客户端真实IP通过请求头的方式传递给微服务
         * 2.在请求头中添加FEIGN_REQUEST的请求头，值为0，标记请求不是Feign调用，而是客户端调用
         */
        ServerHttpRequest request = exchange.getRequest().mutate().
            header(CommonConstants.REAL_IP,exchange.getRequest().getRemoteAddress().getHostString()).
            header(CommonConstants.FEIGN_REQUEST_KEY,CommonConstants.FEIGN_REQUEST_FALSE).
            build();
        return chain.filter(exchange.mutate().request(request).build()).then(Mono.fromRunnable(()->{
            /**
             * post拦截逻辑
             * 在请求执行完微服务之后，需要刷新token在redis的时间
             * 判断token不为空  && Redis还存在这个token对于的key,这时候需要延长Redis中对应key的有效时间。
             */
            String token,redisKey;
            if(!StringUtils.isEmpty(token =exchange.getRequest().getHeaders().getFirst(CommonConstants.TOKEN_NAME))
                && redisTemplate.hasKey(redisKey = CommonRedisKey.USER_TOKEN.getRealKey(token))){
                redisTemplate.expire(redisKey, CommonRedisKey.USER_TOKEN.getExpireTime(), CommonRedisKey.USER_TOKEN.getUnit());
            }
        }));
    }
}
```

```
@Configuration
public class CorsConfig {    cleye, 2022/6/7, 14:42 * init
    @Bean
    public CorsWebFilter corsFilter() {
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource(new PathPatternParser());
        CorsConfiguration config = new CorsConfiguration();
        // 允许cookies跨域
        config.setAllowCredentials(true);
        // #允许向该服务器提交请求的URI，*表示全部允许，在SpringMVC中，如果设成*，会自动转成当前请求头中的Origin
        config.addAllowedOrigin("*");
        // #允许访问的头信息，*表示全部
        config.addAllowedHeader("*");
        // 预检请求的缓存时间（秒），即在这个时间段里，对于相同的跨域请求不会再预检了
        config.setMaxAge(18000L);
        // 允许提交请求的方法，*表示全部允许
        config.addAllowedMethod("*");
        source.registerCorsConfiguration(path: "**", config);
        return new CorsWebFilter(source);
    }
}
```

```
@Configuration
public class GatewayConfiguration {    cleye, 2022/6/7, 14:42 * init
    private final List<ViewResolver> viewResolvers;
    private final ServerCodecConfigurer serverCodecConfigurer;

    public GatewayConfiguration(ObjectProvider<List<ViewResolver>> viewResolversProvider,
                               ServerCodecConfigurer serverCodecConfigurer) {
        this.viewResolvers = viewResolversProvider.getIfAvailable(Collections::emptyList);
        this.serverCodecConfigurer = serverCodecConfigurer;
    }

    // 配置限流的异常处理器
    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    public SentinelGatewayBlockExceptionHandler sentinelGatewayBlockExceptionHandler() {
        // Register the block exception handler for Spring Cloud Gateway.
        return new SentinelGatewayBlockExceptionHandler(viewResolvers, serverCodecConfigurer);
    }

    // 初始化一个限流的过滤器
    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    public GlobalFilter sentinelGatewayFilter() { return new SentinelGatewayFilter(); }

    @PostConstruct
    public void initBlockHandlers() {
        BlockRequestHandler blockRequestHandler = new BlockRequestHandler() {
            public Mono<ServerResponse> handleRequest(ServerWebExchange serverWebExchange, Throwable throwable) {
                Map map = new HashMap<>();
                map.put("code", 0);
                map.put("message", "接口被限流了");
                return ServerResponse.status(HttpStatus.OK)
                    .contentType(MediaType.APPLICATION_JSON)
                    .body(BodyInserters.fromValue(map));
            }
        };
        GatewayCallbackManager.setBlockHandler(blockRequestHandler);
    }

    @PostConstruct
    private void initCustomizedApis() {
        Set<ApiDefinition> definitions = new HashSet<>();
        ApiDefinition api1 = new ApiDefinition( apiName: "order_api1")
            .setPredicateItems(new HashSet<ApiPredicateItem>() {
                add(new ApiPathPredicateItem().setPattern("/order-serv/api1/**").
                    setMatchStrategy(SentinelGatewayConstants.URL_MATCH_STRATEGY_PREFIX));
            });
        definitions.add(api1);
        GatewayApiDefinitionManager.loadApiDefinitions(definitions);
    }

    // 增加对商品微服务的限流
    @PostConstruct
    private void initGatewayRules() {
        Set<GatewayFlowRule> rules = new HashSet<>();
        /* rules.add(new GatewayFlowRule("product route")
            .setCount(3)
            .setIntervalSec(1)
        );
        rules.add(new GatewayFlowRule("order_api1")
            .setCount(1)
            .setIntervalSec(1)); */
        GatewayRuleManager.loadRules(rules);
    }
}
```

五、 运行结果

商品上架

商品上架

欢迎来到叮丁严选

用户名: 李文 我的订单



首页

全部商品

个人中心

我的订单

专属福利

输入关键字

Q

叮丁秒杀

每日特价

大牌闪购

品类秒杀

秒杀

S A L E

10:00 秒杀已结束

12:00 即将开始

14:00 即将开始



Redmi K30 5G 6GB+128GB紫玉幻境
Redmi K30 5G双模 120Hz流速屏 骁龙...

¥1099 ¥1699
已售100%

立即抢购



玉翎 新疆和田玉
观音吊坠男羊脂白玉佛玉坠女本命佛...

¥899 ¥1399
已售100%

立即抢购



夏普70英寸4K+HDR10液晶电视
夏普 (SHARP) 70A5RD 70英寸日本...

¥3999 ¥6399
已售100%

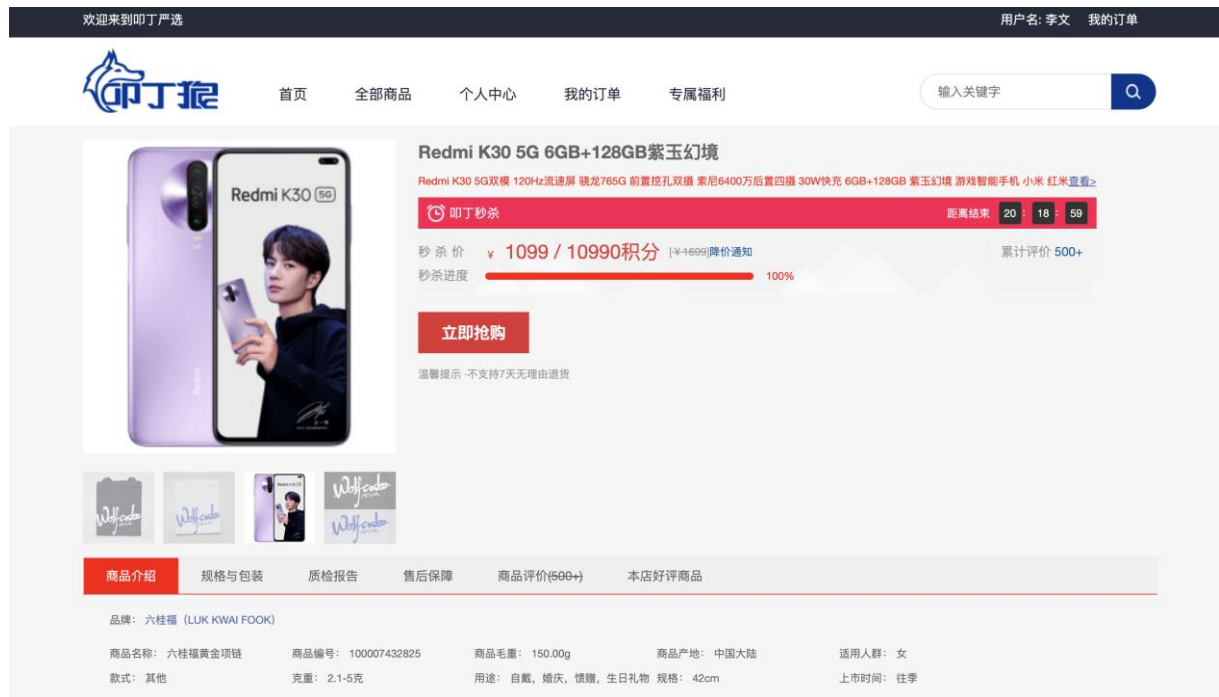
立即抢购



创维10公斤大容量变频滚筒
创维(SKYWORTH) 10公斤 滚筒洗衣...

¥1699 ¥2399
已售100%

立即抢购



六、设计体会

本次课程设计中使用到了 JavaWeb 开发框架 Spring 及其生态，在为期 16 天的时间里快速地搭建了一个秒杀系统，其中的各个微服务互相调用，分布式架构的高可用和高性能都是值得去深入学习的内容。

秒杀具有瞬时高并发的特点，针对这一特点，必须要做限流 + 异步 + 缓存（页面静态化）+ 独立部署。对于这样的服务应该尽可能将 cpu 用在处理业务上，因此日志写入应该交给消息队列进行，消息队列是常用的中间件，大多应用中，可通过消息服务中间件来提升系统异步通信、扩展解耦能力。

七、 参考文献

无

八、 教师评语